

4.1 Introduction

In the previous lecture, we saw that binary prefix-free codes allow to represent a source without errors. But, which is the optimal code in terms of efficiency, i.e., the one that minimizes the average length?

Consider a source U taking values in $\{1, \dots, r\}$ with probability distribution $\{p_1, \dots, p_r\}$. In order to find the binary prefix-free source code that minimizes the average length

$$L_{av} = \sum_{j=1}^r p_j \ell_j, \quad (4.1)$$

if the probabilities are ordered as

$$p_1 \geq p_2 \geq \dots \geq p_r, \quad (4.2)$$

then the optimal code must chose a lengths distribution $\{\ell_1, \ell_2, \dots, \ell_r\}$ such that

$$\ell_1 \leq \ell_2 \leq \dots \leq \ell_r. \quad (4.3)$$

Any other choice results in a suboptimal code in terms of minimizing the expected length! The codes satisfying such condition are **Huffman codes**.

So far, our study of the tree representation of source codes has only taken into account the alphabet of the source and lengths of the resulting codewords. We now incorporate the probabilities of the source in order to derive the optimal source codes. A rooted tree with probabilities is a finite rooted tree with probabilities assigned to each node and leaf such that the probability of a node is the sum of the probabilities of its children nodes, and the root has probability 1. Figure 4.1 illustrates a possible tree with probabilities

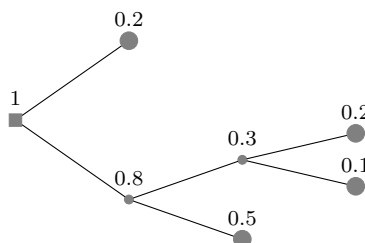


Figure 4.1: Example of a binary rooted tree with probabilities.

4.2 Huffman Codes

Huffman codes, invented by David Huffman while he was a doctoral student at MIT in the 1950s, are the optimal source codes, that can be described using 3 elementary steps. Consider a discrete source taking values in $\{1, \dots, r\}$ with probability distribution $\{p_1, \dots, p_r\}$. To determine a Huffman code:

1. Create r leaves corresponding to the r symbols, and sort them in decreasing probability.
2. Merge the 2 least probable nodes into one new node, labeled with probability sum.
3. Go back to Step 1 until the root is reached.

Let us see an example.

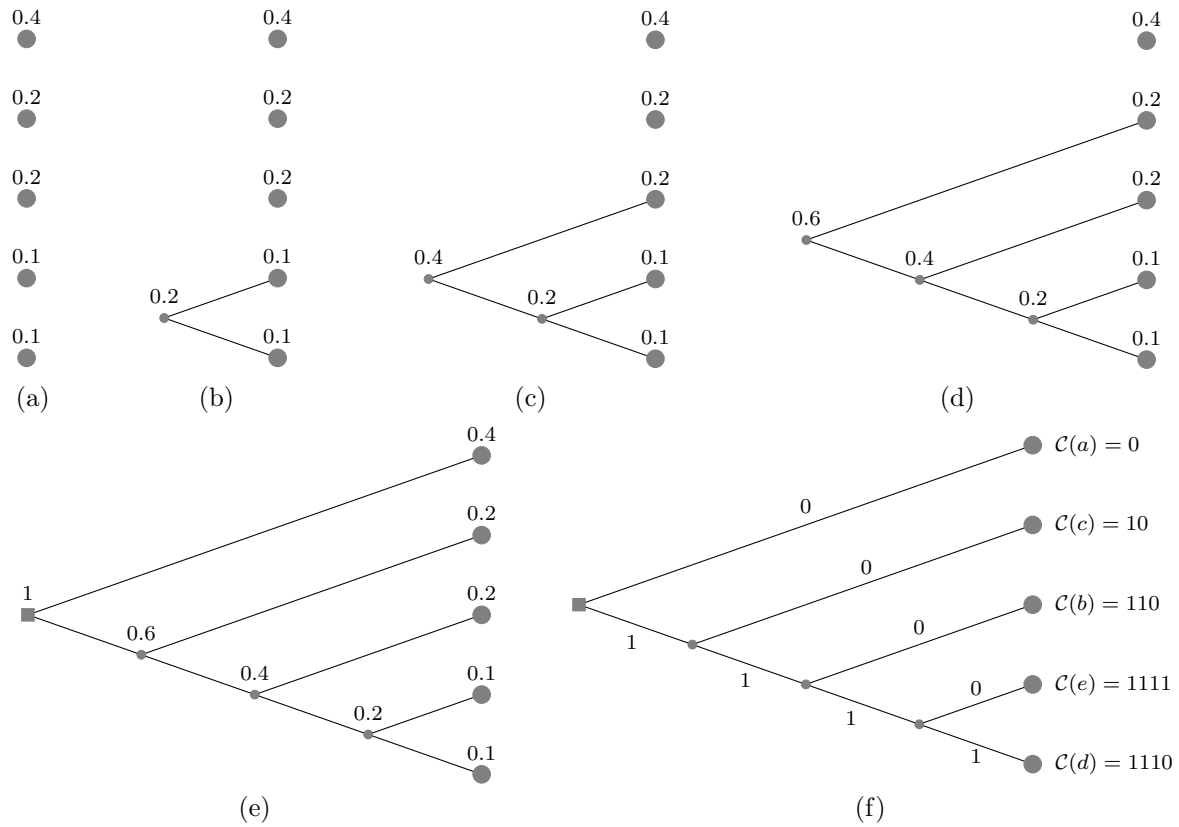


Figure 4.2: Construction of a Huffman code in Example 4.1.

Example 4.1. Design the optimal source code for a source taking values in $\{a, b, c, d, e\}$ with probability distribution $\{0.4, 0.2, 0.2, 0.1, 0.1\}$. We start by writing the $r = 5$ leaves corresponding to the r symbols in decreasing probability, as in Figure Figure 4.2(a). We next merge the 2 least probable nodes into a new node, labeled with probability $0.1 + 0.1 = 0.2$ as in Figure 4.2(b). Now, we repeat the procedure but with the nodes that still have no parent. The 2 least probable nodes are the ones with probability 0.2, hence merged into a new node as in Figure 4.2(c). At this point, we can choose whether to merge the non-parented node of probability 0.2 with either the leaf of probability 0.4 or the node with the same probability created in the previous step. We choose the second option as in Figure 4.2(d), creating a new node of probability 0.6. The last step in Figure 4.2(e) results in the rooted binary tree. It only remains to label the binary tree as we did in the previous lecture and obtain the prefix-free code $\{0, 10, 110, 1110, 111\}$ for the source $\{a, b, c, d, e\}$, as in Figure 4.2(f). The Huffman code that we obtained is prefix-free, hence uniquely decodable, with lengths distribution $\{1, 2, 3, 4, 4\}$. When used to represent the source U , its average length is

$$L_{\text{av}} = 1 \cdot 0.4 + 2 \cdot 0.2 + 3 \cdot 0.2 + 4 \cdot 0.1 + 4 \cdot 0.1 = 2.2 \quad (4.4)$$

bits per symbol. There are points in the algorithm, it is possible to group nodes differently. In this case, the code will be different, but still has the same length!

Problem 4.2. Find a Huffman code for the source in Example 4.1 with a length distribution different from $\{1, 2, 3, 4, 4\}$.

Solution: We follow the same steps as in Figure 4.2(a)–(c), but at the next step, we combine the two leaves of probabilities 0.4 and 0.2 into a new node of probability 0.6, and follow the algorithm to obtain the binary tree in Figure 4.3. The obtained prefix-free code is $\{00, 01, 10, 110, 111\}$, with length distribution $\{2, 2, 2, 3, 3\}$ and, even the code has a length distribution different from the one obtained in Example 4.1, the average length is still

$$L_{\text{av}} = 2 \cdot 0.4 + 2 \cdot 0.2 + 2 \cdot 0.2 + 3 \cdot 0.1 + 3 \cdot 0.1 = 2.2 \quad (4.5)$$

bits per symbol.

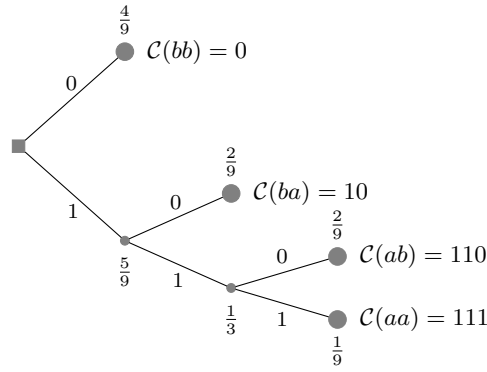


Figure 4.4: Binary tree with probabilities of Problem 4.5.

Problem 4.4. Consider a discrete memoryless source U taking values over $\{a, b\}$ with distribution $\{\frac{1}{3}, \frac{2}{3}\}$. Find the optimal code for U , calculate the average length and check that the inequality (4.13) is satisfied.

Solution: In this case, the resulting Huffman code is either $\{0, 1\}$ or $\{1, 0\}$, of length distribution $\{1, 1\}$ and average length $L_{\text{av}} = 1$ bits per symbol. Since $H(U) = -\sum_{j=1}^r p_j \log_2 p_j = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \approx 0.918296$, we indeed have that $0.918296 \leq L_{\text{av}} < 1.918296$.

Problem 4.5. Consider two consecutive symbols (U_1, U_2) of a discrete memoryless source where each U_i is described as in Problem 4.4. Find the optimal code for (U_1, U_2) , calculate the average length per symbol, and check that the inequality (4.13) is satisfied. Compare your results to that of Problem 4.4.

Solution: We first construct the alphabet and the probability distribution for (U_1, U_2) . The alphabet is the Cartesian product $\{a, b\} \times \{a, b\} = \{aa, ab, ba, bb\}$ and, since symbols are i.i.d., the probability distribution is $\{\frac{1}{9}, \frac{2}{9}, \frac{2}{9}, \frac{4}{9}\}$. After sorting the probabilities and following the Huffman algorithm, we obtain the binary tree in Figure 4.4. A possible Huffman code is then $\{111, 110, 10, 0\}$, of lengths $\{3, 3, 2, 1\}$ and average length

$$L_{\text{av}} = 3 \cdot \frac{1}{9} + 3 \cdot \frac{2}{9} + 2 \cdot \frac{2}{9} + 1 \cdot \frac{4}{9} = \frac{17}{9} \approx 1.88889 \quad (4.14)$$

bits for two source symbols. In this case, the entropy of (U_1, U_2) is given by

$$H(U_1, U_2) = -\sum_{j=1}^r p_j \log_2 p_j = -\frac{1}{9} \log_2 \frac{1}{9} - \frac{2}{9} \log_2 \frac{2}{9} - \frac{2}{9} \log_2 \frac{2}{9} - \frac{4}{9} \log_2 \frac{4}{9} \approx 1.83659,$$

hence (4.13) is satisfied as expected because $1.83659 \leq L_{\text{av}} < 2.83659$. Compared to the Huffman code in Problem 4.4, we observe that the Huffman code $\{111, 110, 10, 0\}$ has average length 1.88889 bits to encode two source symbols, which corresponds to an average length of $\frac{1.88889}{2} = 0.944444$ bits per symbol, slightly smaller than 1.

In Problem 4.5 we have seen that we can do better if we encode blocks of symbols of a discrete memoryless source rather than encoding a single symbol. The benefit increases if we find codes for long source sequences $(U_1, \dots, U_n)$ as $n \rightarrow \infty$. This is the topic of our next lecture.